

L4 Design Process Document — HELEP

1. Project Specification

HELEP (Help Emergency Location & Emergency Platform) is a real-time emergency management system designed for urban safety. When a citizen presses SOS, the system locates the nearest available responder using GPS coordinates, dispatches them, and notifies the citizen — all within one second. Primary users are citizens (trigger SOS), responders (receive assignments), police (monitor analytics), and admins (manage the platform). The business value is reducing emergency response time through automated, event-driven dispatching that eliminates manual radio coordination and human error in responder selection.

2. Requirements Analysis

2.1 Functional Requirements (from SRS §2)

Requirement

- F1 Citizens register with phone, password, and role
- F2 Citizens trigger and cancel SOS with GPS coordinates
- F3 System assigns nearest available responder automatically
- F4 System notifies citizen when responder is assigned
- F5 Responders confirm or reject assignments
- F6 Police view real-time analytics and zone heatmaps
- F7 Citizens manage emergency contacts
- F8 System detects when responder enters a safety zone

2.2 Non-Functional Requirements (SRS §3)

NFR	Measurable Acceptance Criterion
Availability	99.9% uptime; services recover automatically via K8s liveness probes
Reliability	99% of SOS notifications delivered within 1s under 100 req/s; at-least-once Kafka delivery
Scalability	sos-service auto-scales 1→5 replicas under CPU load via HPA
Confidentiality	JWT auth on all protected endpoints; K8s Secrets for JWT_SECRET; NetworkPolicy default-deny
Integrity	bcrypt password hashing; atomic responder assignment via SQLite unique constraint
Usability	REST API with predictable JSON responses; <200ms p99 latency on local cluster
Portability	Each service containerised; Helm chart deploys to any K8s cluster

2.3 Constraints (SRS §4)

Constraint	Architectural Risk
Single responder per incident	Race condition risk → mitigated by atomic SQLite UPDATE WHERE busy=0
Trigger→notify < 1 second	Synchronous HTTP would add latency → mitigated by async Kafka messaging
No real SMS/GPS hardware	Simulated via fake URIs and JSON lat/lon inputs

3. Architectural Drivers & ASRs

The three most architecturally significant requirements are:

ASR-1: Reliability (SRS §3) Every SOS must be delivered and processed exactly once even under failure. This drove the choice of Kafka with manual offset commit (at-least-once delivery), idempotent producer (enable_idempotence=True), and Circuit Breaker pattern in every service's events.py.

ASR-2: Scalability (SRS §3) Emergency spikes are unpredictable. This drove the stateless service design (no shared memory between replicas), Kafka consumer groups (multiple replicas share topic partitions), and HPA on sos-service.

ASR-3: Confidentiality (SRS §3) User identity and location data are sensitive. This drove JWT authentication, bcrypt hashing, Kubernetes Secrets for JWT_SECRET, and NetworkPolicy default-deny between pods.

4. Component Identification

4.1 SRS-listed components (8)

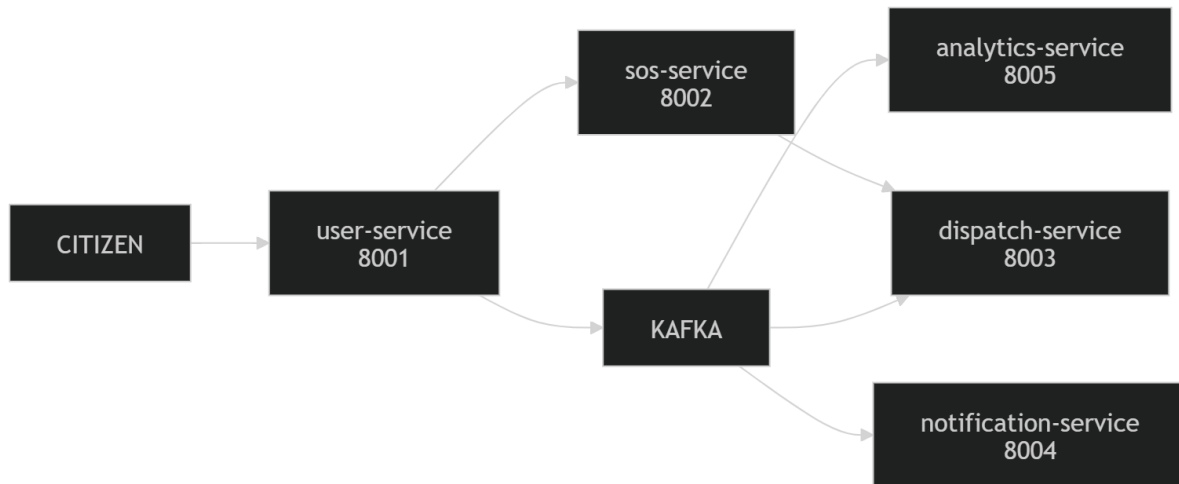
User Management, Emergency Component, Incident Report & Response, Localization, Alert Management, Alert Delivery, Feedback & Review, Analytics & Statistics.

4.2 Service decomposition (5 services)

SRS Components	Service	Justification
User Management	user-service	Single identity authority; JWT issued here only
Emergency Component	sos-service	Owns SOS lifecycle (trigger/cancel); publishes saga start event
Incident Report + Localization	dispatch-service	Both concern responder assignment; splitting would create chatty HTTP calls
Alert Management + Delivery	notification-service	Delivery is pure I/O; isolating it allows independent scaling
Analytics & Statistics	analytics-service	Read-only consumer; isolation prevents analytics load from affecting dispatch

SRS Components	Service	Justification
Feedback & Review	out of scope	Deferred — no ASR depends on it for MVP

Diagram 1 — System Architecture



5. Architectural Style — Choice & Justification

Chosen: Microservices + Event-Driven Architecture (prescribed by SRS)

Alternative 1: Monolith A monolith could satisfy F1–F8 but fails ASR-2 (Scalability) — you cannot scale only the SOS intake without scaling the entire application. It also fails ASR-1 (Reliability) — a single crash takes down all functions simultaneously. Trade-off: simpler deployment vs poor fault isolation.

Alternative 2: SOA (Service-Oriented Architecture) SOA with an ESB (Enterprise Service Bus) satisfies functional requirements but introduces a synchronous central broker — a single point of failure violating ASR-1. The ESB also adds latency, violating the <1s delivery constraint (SRS §4). Trade-off: standardised contracts vs availability risk.

Microservices + Event-Driven wins because: each service fails independently (ASR-1), each scales independently (ASR-2), and Kafka's async messaging keeps end-to-end latency under 1s (SRS §4).

6. Architectural Patterns Applied

Pattern	File	Problem Solved
Choreographed Saga	sos/main.py → dispatch/main.py → notification/main.py	Multi-step emergency workflow without central orchestrator
Pub/Sub	every service's events.py	Decouples producers from consumers; async delivery
Repository	every service's db.py	Isolates DB logic; prevents SQL scattered across handlers
Strategy	dispatch-service/app/matching.py	Pluggable matching algorithms switched via MATCHER env var

Pattern	File	Problem Solved
Outbox-lite	sos-service/app/main.py trigger()	DB write before Kafka publish prevents lost events on crash
Circuit Breaker	every events.py CircuitBreaker class	Stops cascade failures when Kafka is unreachable
Bulkhead	separate Kafka consumer group per service	One slow service cannot block another's event processing
Idempotency Key	dispatch-service/app/db.py assignment PK	Prevents double-dispatch if same event is redelivered

See patterns.pdf for full code citations.

Diagram 2 — Saga Choreography Flow

7. Architecture Decision Records



ADR-001: Kafka partition keying by incident_id

Context: Multiple replicas of dispatch-service consume the same topic. Without ordering guarantees, two replicas could process the same SOS simultaneously and double-assign a responder. **Decision:** Use incident_id as the Kafka partition key in every publish() call. Same-key events always land on the same partition, which is owned by exactly one consumer replica at a time. **Consequences:** Strong ordering per incident. Maximum parallelism equals number of partitions (set to 3). **Alternatives:** Random partitioning — rejected because it breaks the no-double-dispatch invariant.

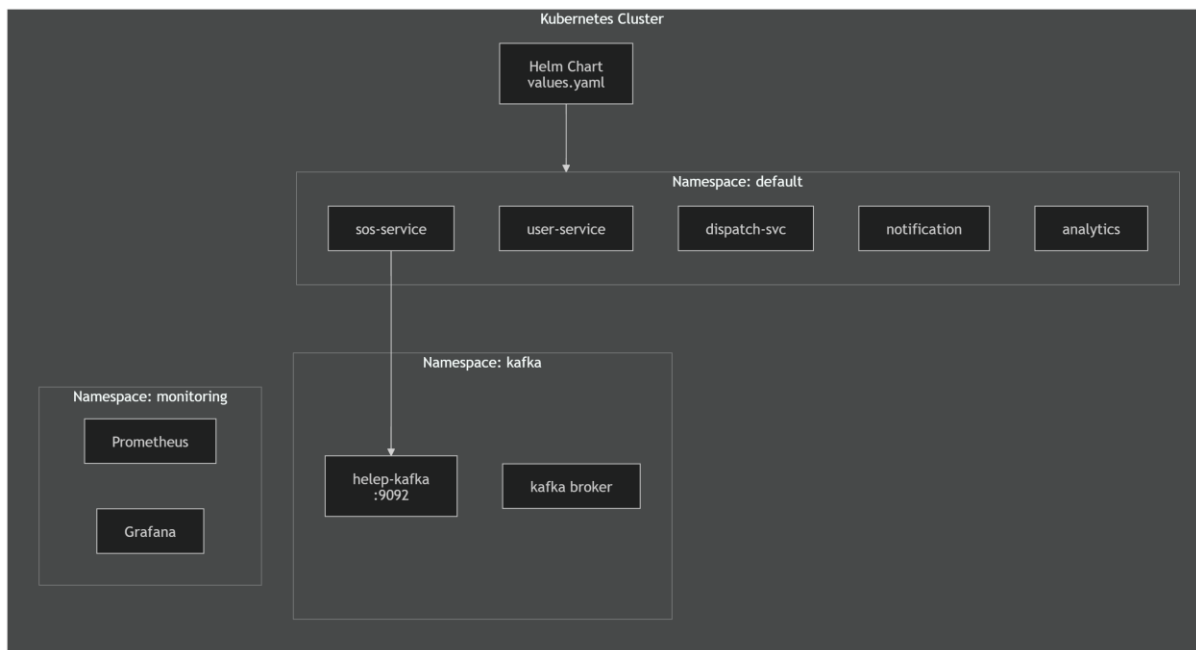
ADR-002: SQLite per service vs shared PostgreSQL

Context: Services need persistent storage. A shared DB is simpler to operate but couples services at the data layer. **Decision:** Each service owns its own SQLite file, mounted via PersistentVolumeClaim in Kubernetes. **Consequences:** Services are fully decoupled — user-service schema changes cannot break dispatch-service. Migration to PostgreSQL per service is straightforward (change DB_PATH env var and driver). **Alternatives:** Shared PostgreSQL — rejected because it violates the microservice principle of independent deployability and creates a single point of failure.

ADR-003: Helm umbrella chart vs separate charts

Context: 5 services need to be deployed together with shared config (JWT_SECRET, Kafka bootstrap address). **Decision:** Single Helm umbrella chart (charts/helep) with shared values.yaml. All services deployed in one helm install command. **Consequences:** Atomic deployment — all services upgrade together. Shared values eliminate config drift between services. **Alternatives:** Separate charts per service — rejected because it requires 5 separate install commands and makes shared secret management complex.

Diagram 3 — Kubernetes Deployment Architecture



8. Trade-offs & Improvement Perspectives

Weakness 1: SQLite is not production-grade SQLite has no concurrent write support. Under high load, dispatch-service will queue writes. Fix: replace with PostgreSQL per service, each running as a K8s StatefulSet with a PVC.

Weakness 2: Single Kafka broker One broker means Kafka is a single point of failure. Fix: configure Strimzi KafkaNodePool with 3 replicas and set `min.insync.replicas=2` for true fault tolerance. This limitation is explicitly acknowledged — production requires ≥ 3 brokers + ≥ 3 controllers for replication and quorum (as noted in project spec).

Weakness 3: No distributed tracing When an SOS fails silently, there is no way to trace which service dropped the event. Fix: add OpenTelemetry instrumentation to each service and deploy Jaeger as a trace collector in the monitoring namespace.